

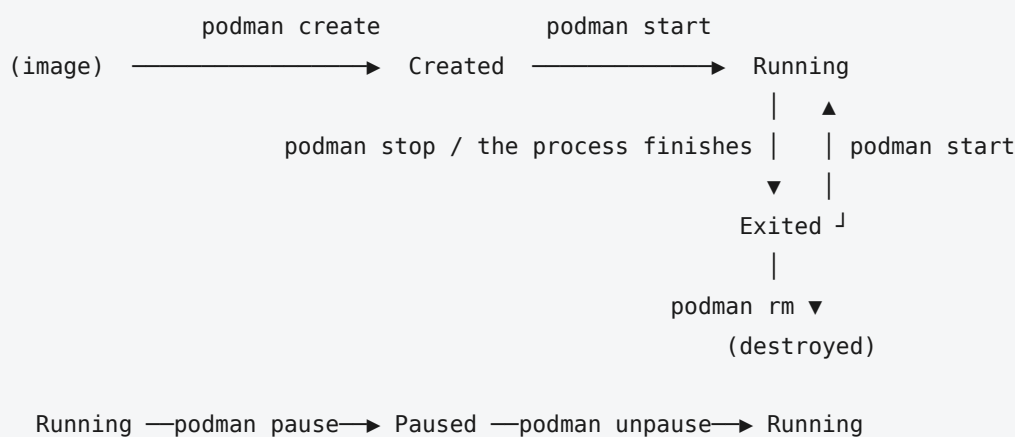
Lab 03 — The life cycle of a container

Theory — birth, life, signals and death of a container; why PID 1 changes everything, and who restarts your containers when there is no daemon.

Objectives

- Know the states of a container and the commands that move it from one to another.
- Understand why a container "stops on its own".
- Master the relationship between the **main process**, **signals** and the **exit code**.
- Choose between `exec` and `attach`, between foreground and background.
- Know what a *restart policy* really does — and what it cannot do without a daemon.

1. The states



Command	Effect
<code>podman create</code>	Prepares the container (writable layer, config) without starting it
<code>podman start</code>	Starts the main process
<code>podman run</code>	<code>create</code> + <code>start</code> (+ <code>pull</code> if the image is missing)
<code>podman stop</code>	Politely asks for a stop, then forces after a delay
<code>podman kill</code>	Forces the stop immediately
<code>podman restart</code>	<code>stop</code> then <code>start</code>
<code>podman pause</code>	Freezes the processes (cgroup <i>freezer</i>), without stopping them
<code>podman rm</code>	Destroys the container and its writable layer

An `Exited` container is not dead: it keeps configuration, writable layer and logs. It can be inspected, restarted, files can be extracted from it. Only `rm` destroys.

2. The fundamental rule: a container lives as long as its PID 1

REMEMBER

A container stops exactly when its **main process** ends. Not before, not after. There is nothing else to understand.

That explains almost every "my container stops on its own":

- `podman run alpine` exits immediately — the default command is `/bin/sh`, which, with no terminal attached, reads nothing and exits.
- `podman run nginx` stays alive — nginx runs in the foreground and never returns.
- A script that puts a service in the **background** (`java -jar ... &`) dies at once: the script ends, so does PID 1.

Hence a design rule: **an image launches its service in the foreground**. No daemon, no `systemd`, no `nohup` inside a container: the engine plays the role of the service manager.

→ LINUX

On a normal Linux machine, PID 1 is `init` (nowadays `systemd`): the first process created by the kernel, ancestor of all others. The kernel treats it specially: if it dies, the system halts; and it **ignores signals by default** for which it installed no handler, so that a clumsy `kill` does not bring the machine down. Inside a container, *your application* inherits that `init` status — with its privileges and its traps.

Corollary: a container is made for **one main process**. Putting the API and the database in the same container breaks the model — they can no longer be restarted, monitored or scaled separately.

3. Foreground, background, and the `-it` duo

```
podman run nginx                # foreground: the terminal is blocked, logs shown
podman run -d nginx              # detached: returns the prompt, prints the container ID
podman run -it alpine sh         # interactive: you get a usable shell
podman run --rm alpine date      # one-off execution, container removed on exit
```

`-it` is misunderstood because these are **two distinct options**: `-i` keeps standard input open — without it a shell sees its `stdin` closed and exits at once; `-t` allocates a pseudo-terminal — the prompt, key echo, `Ctrl+C`. In scripts or CI, `-i` alone; for human use, `-it`.

PITFALL

In the foreground, `Ctrl+C` sends `SIGINT` to the container's process: nginx stops. In `-it` mode, the sequence `Ctrl+P` then `Ctrl+Q` instead lets you **detach without stopping** the container.

PODMAN

With Docker, "detached" means the daemon keeps the container. With Podman there is no daemon: when `podman run -d` returns, it is `common` that stays — a process of a few hundred KB, one per container, that keeps the `stdout` / `stderr` pipes open, writes the logs, and records the exit code when PID 1 dies. You will see it in `ps` right above your containers. If you close your WSL session, `common` and your containers die with it... unless `systemd` holds them (section 6).

4. Signals: how a container dies

`podman stop` is not a switch. It runs a protocol:

1. Send **SIGTERM** to PID 1: "terminate cleanly".
2. Wait for a **grace period**, 10 seconds by default (`-t` to change it).
3. If the process is still there, send **SIGKILL**, uncatchable, immediate — Podman announces it:
`StopSignal SIGTERM failed to stop container ... in 10 seconds, resorting to SIGKILL`.

`podman kill` jumps straight to step 3. And `podman rm -f` does a full `stop`, 10 seconds included — hence the `-t 0` of lab 01.

→ LINUX

A **signal** is an asynchronous notification the kernel delivers to a process: **SIGTERM** (15) asks for a stop and can be caught, **SIGKILL** (9) kills without appeal, **SIGINT** (2) is `Ctrl+C`. A program "handles" a signal by installing a *handler*; otherwise the default action applies — for **SIGTERM**, dying. Except for PID 1, which has no default action: it ignores.

During those 10 seconds, a well-written Spring Boot application finishes in-flight requests, closes the PostgreSQL pool, unregisters from service discovery. With **SIGKILL**, none of that: requests cut, connections left hanging on the database side, possibly inconsistent data.

→ JAVA / SPRING BOOT

The JVM turns **SIGTERM** into the execution of **shutdown hooks** (`Runtime.addShutdownHook`). Spring Boot registers one that closes the context: `@PreDestroy`, closing the JDBC pool, stopping the web server. With `server.shutdown=graceful` (and `spring.lifecycle.timeout-per-shutdown-phase=20s`), the server stops accepting connections and lets in-flight requests finish. All of that **assumes** **SIGTERM** **reaches** the JVM.

Two traps that prevent the signal from being received:

1. A shell in front of the application. That is the case of a start-up script that launches the application without `exec`, or the *shell* form of a `CMD` (`CMD java -jar app.jar` becomes `/bin/sh -c "java -jar app.jar"`). PID 1 is then `sh`, which **does not forward** **SIGTERM** to its child: Java never receives the signal, waits 10 seconds, then is killed. The remedy is twofold: `exec` form (`CMD ["java", "-jar", "app.jar"]`) and, in a script, `exec java -jar app.jar` as the last line. This syntax detail, covered in lab 04, decides the quality of your shutdowns.

2. The special status of PID 1. A process that does not handle **SIGTERM** and runs as PID 1 is **insensitive** to `podman stop`, then killed after the delay. PID 1 must also "adopt" orphans, otherwise *zombies* accumulate. Hence `--init`, which inserts a proper mini-init (`podman-init`) in front of your application.

5. Exit codes

```
podman run --rm alpine sh -c 'exit 3'; echo $?      # 3
podman ps -a --format 'table {{.Names}}\t{{.Status}}'
```

The container's exit code is that of its PID 1, kept in its status (`Exited (3)`). A few codes to recognise:

Code	Usual meaning
0	Normal termination
1	Generic application error
125	The engine itself failed (invalid option)
126	Command found but not executable (or <code>podman</code> could not open the port)
127	Command not found in the image
137	Killed by <code>SIGKILL</code> (128+9) — <code>podman kill</code> , end of the grace period, or the OOM killer
143	Terminated by <code>SIGTERM</code> (128+15) — a clean <code>stop</code>

137 is the code you will see most in production: a `stop` past the grace period, or a memory limit exceeded. `podman inspect` settles it: `.State.OOMKilled` is `true` in the second case.

6. Restart policies — and who enforces them

```
podman run -d --restart=unless-stopped --name api my-api:1.0
```

Policy	Behaviour
<code>no</code> (default)	No automatic restart
<code>on-failure[:N]</code>	Restarts if the exit code is $\neq 0$, at most N times
<code>always</code>	Always restarts, including after a host reboot... if someone is there to do it
<code>unless-stopped</code>	Like <code>always</code> , unless you stopped it manually

With Docker, the daemon enforces these rules, including at machine start-up. With Podman, `common` restarts the container as long as your session lives — but after a reboot, nobody is there to read the policy.

PODMAN

Podman's answer is **systemd**, Linux's service manager, through **Quadlet**: a ten-line file `~/.config/containers/systemd/api.container` (`[Container]`, `Image=`, `PublishPort=` ...) and `systemctl --user start api`. The container becomes an ordinary service: start at boot, restart on failure, logs in `journalctl`. That is why Podman did not want a daemon: Linux's already exists. Put into practice in lab 10.

PITFALL

`always` restarts a container even after a manual `stop`, at the next start of the engine. `unless-stopped` remembers your intention: it is almost always the right choice on a single machine.

7. Observe and intervene

```
podman logs -f --tail 50 api      # output stream of PID 1
podman exec -it api sh            # new process INSIDE the container
podman attach api                 # reconnect to the existing PID 1
podman top api                    # the container's processes, seen from the host
podman stats api                  # CPU/memory consumption in real time
podman inspect api                # complete state, JSON
podman cp api:/app/log.txt .      # extract a file, even from a stopped container
podman events --since 10m         # the journal of creations, stops, deaths
```

`exec` **creates a new process** in the container's namespaces — that is what you want to go and see what is happening. `attach` reconnects you to the input/output of the **existing PID 1**: a `Ctrl+C` there stops the container.

`podman logs` only shows what PID 1 wrote to `stdout` / `stderr`, captured by `common`. An application that writes to a file will not show up — hence the rule: **log to standard output**. That is Spring Boot's default; so do not configure a `logging.file.name`.

8. In the workplace

- The Spring Boot back end runs with `-d`, with `--restart=unless-stopped` under Docker or as a Quadlet service under Podman — or with no policy under an orchestrator, which handles restarts itself.
- Clean shutdown is a production topic: `SIGTERM` received + Spring *graceful shutdown* = deployments without a lost request.
- Diagnosis always follows the same sequence: `ps -a` (status, code), `logs` (what the application said), `inspect` (OOM? configuration?), then `exec` if the container is still alive.

Remember

- A container lives exactly as long as its main process (PID 1).
- Services must run **in the foreground**: no daemon, no `&`, no `systemd` inside the container.
- `-i` keeps `stdin` open, `-t` allocates a terminal. `stop` = `SIGTERM`, grace period, then `SIGKILL` (Podman warns); `kill` = direct `SIGKILL`; `rm -f` = a full `stop` without `-t 0`.
- The `exec` form (`["java", "-jar", "x.jar"]`) is indispensable to receive signals.
- `137` = killed (KILL or OOM), `143` = stopped cleanly (TERM), `127` = command not found.
- `rm` destroys the container's data; `stop` does not. `exec` creates a process, `attach` hooks onto PID 1. Without a daemon, restart at boot goes through `systemd` (Quadlet).

Vocabulary

PID 1: the container's main process. — **grace period**: time between `SIGTERM` and `SIGKILL`. — **graceful shutdown**: clean stop that finishes work in progress. — **restart policy**: automatic restart rule. — **OOM killer**: the kernel kills a process when memory runs out. — **zombie**: terminated process whose exit code nobody read. — **common**: supervisor of a Podman container. — **Quadlet**: Podman's integration with `systemd` (`.container` files).